Redes Neuronales y Aprendizaje Profundo

Curso de Deep Learning

Universidad EAFIT

Laboratorio # 2 – Núcleo II (Valor: 10%)

Parte A -Transfer Learning con PyTorch

Objetivos

- Entender el concepto de transfer learning y cuándo usarlo.
- Cargar y usar modelos preentrenados con *torchvision.models* usando la API moderna.
- Extraer *features* con una red convolucional fija y entrenar un clasificador sobre ellas.
- Reemplazar el clasificador final y hacer *fine-tuning* por etapas.
- Comparar el comportamiento de VGG-16 y ResNet-18 adaptados a un problema binario
- Visualizar y comparar experimentos con *TensorBoard*.

Descripción de los modelos a desarrollar para el laboratorio

	Modelo	Descripción
M1	VGG-16	VGG-16 como extractor de features fijo + clasificador lineal.
M2	VGG-16	VGG-16 con clasificador reemplazado y fine-tuning completo.
M3	ResNet-18	ResNet-18 adaptado a Cats vs. Dogs, propuesto por el estudiante

Notebook Lab_2_TransferLearning_Workshop.ipynb

Actividad 1 — Dataset y pipeline de preprocesamiento

Usar el dataset Kaggle Cats vs. Dogs (https://www.kaggle.com/datasets/kunalgupta2616/dog-vs-cat-images-data). Una vez descargado y extraído, debes verificar y eliminar imágenes corruptas con check_image_dir. Luego construye el pipeline de transforms para preprocesar las imágenes correctamente.

Los modelos preentrenados en ImageNet esperan imágenes de 224×224 normalizadas con media [0.485, 0.456, 0.406] y desviación estándar [0.229, 0.224, 0.225]. El pipeline debe seguir el orden: Resize a 256 → CenterCrop a 224 → ToTensor → Normalize.

Pregunta: ¿De dónde vienen esos valores de media y std? ¿Por qué es importante usarlos si vas a cargar pesos preentrenados?

Actividad 2 — Modelos preentrenados y extracción de features

Carga VGG-16 con pesos preentrenados en ImageNet usando la API moderna de torchvision. Verifica que la salida del modelo para una imagen de muestra tenga shape [1, 1000], luego usa el mapeo de clases de ImageNet para mostrar el nombre de la clase predicha.

Luego usa vgg.features como extractor fijo para pre-computar los vectores de features de 800 imágenes del dataset. Almacénalos en un tensor de shape [800, 512×7×7] usando torch.no_grad() para no calcular gradientes durante la extracción.

Pregunta: ¿Cuántos parámetros tiene VGG-16 en total? ¿Qué hace la capa Dropout y por qué mejora la generalización? ¿Por qué la última capa tiene 1000 salidas?

Actividad 3 — Clasificador lineal sobre features precomputadas

Con los features fijos, entrena solo una capa lineal que mapee de 512×7×7 entradas a 2 clases. Este es M1 en su primera etapa.

Ten en cuenta que usar nn.LogSoftmax() al final de la red junto con CrossEntropyLoss es un error: CrossEntropyLoss en PyTorch ya aplica softmax internamente, combinándolos se doble-aplica la operación y produce resultados incorrectos. Define net como una sola capa nn.Linear sin ninguna activación al final. Entrena por 5 épocas y guarda la curva con TensorBoard.

Pregunta: ¿Cuál es la diferencia entre CrossEntropyLoss y NLLLoss + LogSoftmax? ¿Por qué combinarlos es un error?

Actividad 4 — Transfer Learning: reemplazar el clasificador y congelar features

Recarga VGG-16 limpio y reemplaza su clasificador final (vgg.classifier) por una capa nn.Linear de 25088 entradas a 2 salidas. Luego congela todos los parámetros de vgg.features poniendo requires_grad = False en cada uno, de modo que solo el nuevo clasificador se entrañe.

Verifica con summary() que el número de parámetros entrenables sea aproximadamente 50k. Entrena por 1 época sobre el dataset completo y guarda el modelo.

Nota: el summary() del código base usa tamaño 244 en lugar de 224. Corrígelo antes de ejecutarlo.

Pregunta: ¿Por qué es importante congelar las capas convolucionales antes de entrenar el nuevo clasificador?

Actividad 5 — Fine-tuning completo

Carga el modelo guardado en la actividad anterior y descongela los parámetros de vgg.features poniendo requires_grad = True. Entrena por 1 época adicional con un learning rate más pequeño (1e-4) para no destruir los pesos preentrenados.

Compara las curvas de esta etapa con las de la etapa congelada en TensorBoard.

Actividad 6 — Adaptar ResNet-18 (M3)

ResNet-18 está disponible en torchvision.models. A diferencia de VGG, su clasificador final se llama fc en lugar de classifier. Congela todos los parámetros de la red, investiga cuántas entradas tiene resnet18.fc, y reemplázalo con una capa lineal hacia 2 clases. Entrena solo el clasificador por 2 épocas y compara con VGG en velocidad y accuracy.

Pregunta: ¿Qué es un residual block? ¿Qué problema resuelve en redes muy profundas? ¿ResNet-18 entrena más rápido que VGG? ¿Por qué?

Preguntas de investigación: Responde en celdas Markdown apoyándote en la documentación, papers o experimentos propios.

1. ¿Cuándo NO conviene usar transfer learning? Da al menos dos escenarios concretos.
2. ¿Qué transforms de data augmentation añadirías al pipeline de entrenamiento para mejorar generalización? Impleméntalos y experimenta.
3. Investiga StepLR y CosineAnnealingLR. Agrega uno al loop de fine-tuning de la Actividad 5 y compara curvas en TensorBoard.
4. ¿Por qué ResNet usa BatchNorm y VGG no? ¿Qué efecto tiene en el entrenamiento?
5. Normalización ImageNet: ¿De dónde vienen exactamente los valores mean=[0.485, 0.456, 0.406] y std=[0.229, 0.224, 0.225]? ¿Qué pasaría si usaras otros valores?
6. CrossEntropyLoss vs NLLLoss: Investiga la diferencia entre nn.CrossEntropyLoss, nn.NLLLoss y nn.LogSoftmax. ¿Por qué no debes añadir LogSoftmax a la red si ya usas CrossEntropyLoss?

7. Learning rate scheduling: Investiga torch.optim.lr_scheduler.StepLR y CosineAnnealingLR. Agrega uno al loop de fine-tuning y compara las curvas.
8. Reto opcional: Usa EfficientNet-B0 (disponible en torchvision.models) y compara accuracy final y tiempo de entrenamiento contra ResNet-18 y VGG-16.

Entregables

Archivo — Notebook (grupoX_LAB#2A.ipynb)

Un único notebook .ipynb con todo el código completo y ejecutado, que incluya las respuestas a todas las preguntas en **celdas Markdown.**

El reporte debe ser autocontenido: alguien que no ejecute el notebook debe poder entender la solución planteada.